Upgrading MV-Table (& MV-Card) for Dynamic Height Feature Implementation

Overview

Our MVTable implementations now have the ability to set the table's height dynamically based on various factors within the page. Prior to the update, MVTables are implemented with a static height (through the `tableHeight` prop), which raises UX/UI concerns where users may have various screen sizes anad further variabilities are introduces when the application is used within OpenFin / W1. This page will outline the process in which we can update our implementations of the MVTable component within an MFE.
NB: Alongside this upgrade, MV-Card is also has changes regarding the layouts as well as adding a classname so that it could be referenced in the dynamic height calculations.

Proposed Behaviour

The following scenarios and the MVTable's behaviour were considered during the upgrade:

1. If the total amount of data rows can fit within the available space (i.e we have enough space to fit all data)
    - The Table will render a height that fits the data rows
    - There will be no excess white space that stretches MVCard
    - Vertical scrollbar will not be introduced
2. If the data exceeds the available space (i.e there is too much data to fit on the page)
    - The Table will fill the rest of the available space
    - The Table's height will take into account the Card's margins
    - Vertical scrollbar **will not be** introduced for the **page**
    - Vertical scrollbar **will be** introduced for **MVTable** to access all data
3. If the available height is less than the minHeight (i.e there is too much content on the page to include the smallest version of the table)
    - Vertical scrollbar **will be** introduced for **both MVTable and the page**

- The actual **Height** of table is set by the following algorithm. (In below code, agGridTotalHeight = header height + row count * row height; availableHeight = page height - other content height; Max_MinHeight = 600px)

```
export function getGridHeight(  agGridTotalHeight: number,   minHeight: number,  availableHeight: number): number {
  let gridHeight = 0;

  // example: agGridTotalHeight: 120, availableHeight: 700, then gridHeight = minHeight(135)
  if (agGridTotalHeight < minHeight) {
     gridHeight = minHeight;
  }

  // example: agGridTotalHeight: 150, availableHeight: 700, then gridHeight = 150
  // example: agGridTotalHeight: 150, availableHeight: 140, then gridHeight = 150
  // example: agGridTotalHeight: 400, availableHeight: 500, then gridHeight = 400
  else if (agGridTotalHeight < Max_MinHeight) {
     gridHeight = agGridTotalHeight;
  }

  // example: agGridTotalHeight: 700, availableHeight: 300, then gridHeight = 600
  else if (availableHeight <= Max_MinHeight) {
     gridHeight = Max_MinHeight;
  }

  // example: agGridTotalHeight: 700, availableHeight: 800, then gridHeight = 700
  else if (  availableHeight > Max_MinHeight &&  agGridTotalHeight <= availableHeight ) {
     gridHeight = agGridTotalHeight;
  }

  // example: agGridTotalHeight: 850, availableHeight: 750, then gridHeight = 750
  else if (  availableHeight > Max_MinHeight &&  agGridTotalHeight > availableHeight ) {
     gridHeight = availableHeight;
  }

  return gridHeight;
}
```

Changes

MVCard

MVCard has relatively simple changes:

- `justify-start` : (previously centred) keeps the card and it's contents left aligned
- `mv-card-container :` this is a class name utilised by MVTable to help with **dynamic height calculations**
- `m-${margin}` : 18px (`margin = 3`) margin all around (previously auto/24px) - this is so that we are able to maintain the left aligned card which helps with width resizing scenarios
- `border border-border` : 1px **grey(GEL: border)** border all around (previously 1px red border top)

MVTable

MVTable's changes we're more substantial. But simply in order:

1. MVTable will check the total amount of data, the available space in the implemented page for the table, and determine whether the height needs to be
    a. Based off of the data
    OR
    b. Based off of the available space
2. MVTable will take in multiple heights factors into a calculation including MVCard and the available height within the viewport given the various components other than the Table
3. The calculation will then pass a height to MVTable and MVTable will render.

Implementation within an MFE (examples from Account MFE)

To implement/ upgrade our MVTable, we need to first ensure we have the appropriate versions:

- **NB:** MVTable and MVCard implementation is in **v1.224.8 and v1.221.1 respectively.**
- install these into the ui folder's `package.json` .
- Ensure that SPA also has the appropriate MVCard package version

We require 2 props passed into MVTable, and a single div tag surrounding the MVTable:

- The props:
    - `containerRef` (optional): this is the container ref of the table. When provided, the dynamic height algorithm will run; otherwise, it will not run and the tableHeight (default as 600px) should be set as fixed height.
    - `minHeight` (optional and default as **135px**) :
        - The minimal minHeight is typically determined by the height of the table when no records are found (i.e the no records found banner will show). This will include the header, meaning if your header wraps to a second line, this may change your minHeight. From Channels, **single-line headers** (e.g TransactionView.tsx) usually have a minimal minHeight of **135px (default value)**, while wrapped / **two-line headers** (RenderAccountList.tsx) usually have a minimal minHeight of **153px**.
        - The maximal minHeight is hardcoded as **600px**.

- The `<div>` tag:
    - We need to wrap a `<div>` tag around our MVTable render.
    - this `<div>` tag will need to have a `ref` that is passed in to the `containerRef` prop of MVTable
- Example (Note: we may have previously passed tableHeight, and we will need to replace this prop):

```
<div data-testid="statement-list-mvtable" ref={tableContainerRef}>
  <MVTable
    rowData={list}
    defaultColumnDef={defaultColumnDef}
    columnDefs={columnDefs}
    enableCellTextSelection
    onColumnPinned={onColumnPinned}
    noRowMessage={noRowMessage}
    getMainMenuItems={getMainMenuItems}
    onFilterChanged={onFilterChanged}
    suppressContextMenu
    className="mt-md"
    columnMenu="legacy"
    suppressHorizontalScroll={false}
    containerRef={tableContainerRef}
  />
</div>
```

Static MVTable

There may be instances where a static MVTable is required. This can be done simply by passing `tableHeight` and the desired value(px) instead of the four props:

```
<div ref = {tableContainerRef}>
  <MVTable
    rowData={allArrangements}
    defaultColumnDef={defaultColumnDef}
    columnDefs={columnDefs}
    onCellClicked={onCellClicked}
    onColumnPinned={onColumnPinned}
    rowClassRules={rowClassRules}
    getMainMenuItems={getMainMenuItems}
    getContextMenuItems={getContextMenuItems as any}
    noRowMessage={noRowMessage}
    className={`${showMaxLoadInfoMessage ? '' : 'mt-2.5'}`}
    onGridReady={onAccountListGridReady}
    onFilterChanged={onFilterChanged}
    columnMenu="legacy"
    onCellKeyDown={onCellKeyDown}
    autoSizeStrategy={{
      type: 'fitCellContents',
      skipHeader: true,
      colIds: ['availableBalance', 'currentBalance', 'status'],
    }}
    ref={mvTableRef}
    tableHeight={300}
  />
</div>
```

Notes

- This dynamic height algorithm works when the accordion is collapsed or window resizes. In these cases, the table can expand its height.
- In case the table cannot expand automatically (rare), make sure the filter accordion and the mv-table are inside the "mv-card-container" div like below.

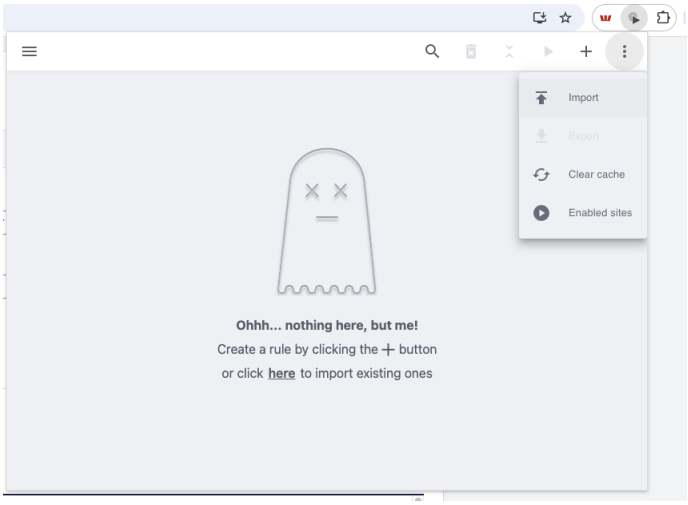
```
<div className="mv-card-container">
    <TransactionSearchForm />

    <TransactionView />

</div>
```

Testing with Tweak

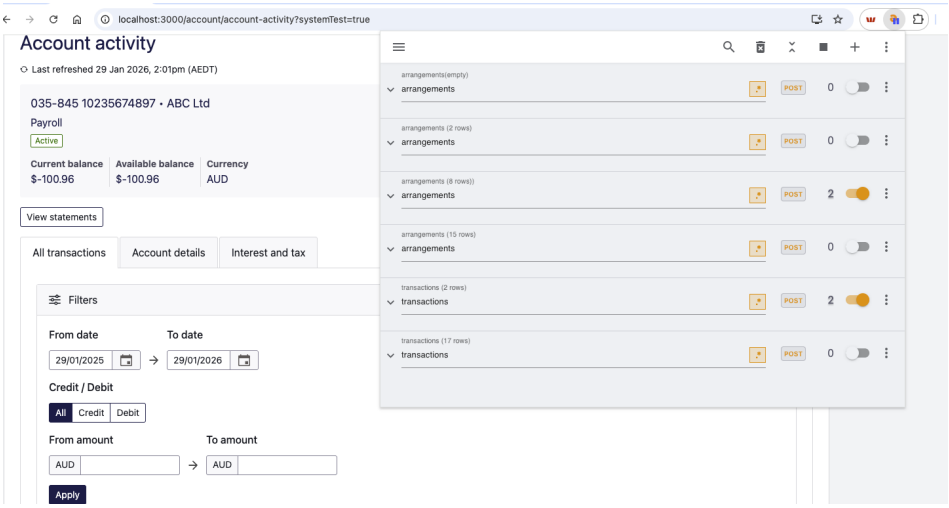
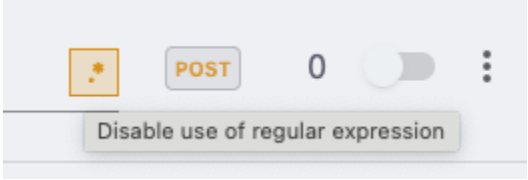
- Install Tweak extension for Chrome through "chrome web store" https://chromewebstore.google.com/detail/tweak-mock-and-modify-htt /feahianecghpnipmhphmfgmpdodhcapi?hl=en-GB&utm_source=ext_sidebar

- Download the below Mock "tweak_2026-02-02T00_05_19.944Z.json" file.

- Import: Click the "import" menu and select the above Mock json file.



- Run it with your Mock response.



Note: Please use **regular expression** in url input box.